

LAPORAN PRATIKUM PEKAN 5

STRUKTUR DATA

Single Linked List



Disusun oleh:

Jose Moniz de Araujo 2511538002

Dosen pengampu:

Wahyudi Dr. S.T.M.T

Asisten Pratikum:

Sherly Sukmadira

DEPARTAMENT INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

TAHUN

2026

KATA PENGANTAR

Puji Syukur penulis penatkan ke hadirat tahun yang Maha Esa atas terselesaikan laporan pekan ke-4 ini. Laporan ini di susun untuk memenuhi tugas mata kuliah Struktur Data yang berfokus pada pemahaman konsep dasar **Single linked List**.

Pada pekan ini, utama pembelajaran adalah memahami bagaimana Struktur Data dapat didefinisikan secara logis melalui Single Linked List, sebelum diimplementasikan ke dalam kode program. Penulis menyadari bahwa laporan ini masih jauh dari sempurna, oleh karena itu kritik dan saran yang membangun sangat diharapkan.

Ahir kata, semoga laporan ini dapat bermanfaat bagi penulis khususnya, dan pembaca pada umumnya.

Padang, 07 Mei 2026

Jose Moniz de Araujo

BAB I

PENDAHULUAN

1.1. Latar belakang

Dalam ilmu komputer, struktur data adalah elemen penting yang digunakan untuk mengatur dan menyimpan data dengan cara yang efisien. Salah satu struktur data yang sering dipakai adalah linked list, berbeda dari array karena tidak menggunakan memori yang berdampingan, melainkan terdiri dari sekumpulan node yang saling terhubung.

Single linked list merupakan tipe Linked List paling sederhana, di mana setiap nodenya hanya memiliki dua komponen, yaitu data dan pointer (referensi) yang mengarah ke node selanjutnya. Struktur ini memungkinkan penambahan dan penghapusan data dilakukan secara fleksibel tanpa perlu memindahkan elemen lain, seperti pada array.

Dalam penerapannya, single linked list memiliki beberapa operasi utama seperti penambahan node (insert) seluruh node. Struktur ini bermanfaat dalam banyak aplikasi, seperti pengelolaan data yang dinamis, sistem antrian, dan pengolahan data yang sering kali berubah.

Namun, penggunaan Single Linked List juga memiliki Kelemahan, seperti tidak adanya akses langsung ke elemen tertentu (harus dilacak dari awal) serta memerlukan memori tambahan untuk menyimpan pointer. Jadi, pemahaman mengenai konsep dan penerapan Single Linked List sangat penting Bagi mahasiswa untuk belajar tentang efisiensi algoritma dan pengelolaan memori.

1.2. Tujuan pratikum

1. Memahami konsep dasar dari single-linked list sebagai salah satu struktur data dinamis.
2. Mengetahui perbedaan antar linked List dengan struktur data lain seperti array.
3. Mampu melakukan operasi dasar pada single Linked List seperti : penambahan (insert), penghapusan (delete), pencarian (search) dan penelusuran (traversal)
4. Memahami kelebihan dan kekurangan penggunaan Single Linked List dalam pengolahan data.

5. Memahami kelebihan dan kekurangan penggunaan Single linked List dalam pengelolaan data.
6. Melatih kemampuan berpikir logis dan pemecahan masalah dalam ilmu komputer.

1.3. Manfaat pratikum

1. Meningkatkan pemahaman tentang konsep Single linked List sebagai struktur data dinamis.
2. Membantu mahasiswa memahami cara kerja Linked List dalam pengelolaan data secara efisien.
3. Melatih kemampuan dalam mengimplementasikan struktur data menggunakan bahasa pemrograman Java.
4. Membantu memahami kelebihan dan kekurangan Single Linked List dibandingkan dengan struktur data lainnya.

BAB II

PEMBAHASAN

2.1 Kode program pratikum pekan 5

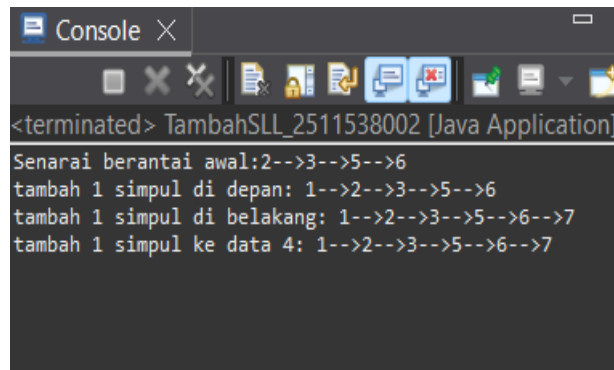
2.1.1 NodeSLL_2511538002

2.1.2 TambahSLL_2511538002

1. Ulangi pembuatan Class seperti program sebelumnya dan beri nama Class tersebut “TambahSLL_2511538002”. Lalu masukan syntax seperti berikut.

```
1 package pekan5_2511538002;
2
3 public class TambahSLL_2511538002 {
4     public static NodeSLL_2511538002 insertAtFront(NodeSLL_2511538002 head_8002, int value_8002) {
5         NodeSLL_2511538002 new_node = new NodeSLL_2511538002(value_8002);
6         new_node.next_8002 = head_8002;
7         return new_node;
8     }
9     // fungsi menambahkan node di akhir SLL
10    public static NodeSLL_2511538002 insertAtEnd (NodeSLL_2511538002 head_8002, int value_8002) {
11        // buat sebuah node dengan sebuah nilai
12        NodeSLL_2511538002 newNode = new NodeSLL_2511538002(value_8002);
13        // jika list kosong maka node jadi head
14        if (head_8002 == null) {
15            return newNode;
16        }
17        // simpan head ke variabel sementara
18        NodeSLL_2511538002 last = head_8002;
19        //keluarkan ke node akhir
20        while (last.next_8002 != null) {
21            last = last.next_8002;
22        }
23        // ubah pointer
24        last.next_8002 = newNode;
25        return head_8002;
26    }
27    static NodeSLL_2511538002 GetNode (int data_8002) {
28        return new NodeSLL_2511538002(data_8002);
29    }
30    static NodeSLL_2511538002 insertPos (NodeSLL_2511538002 headNode_8002, int position_8002, int value_8002) {
31        NodeSLL_2511538002 head_8002 = headNode_8002;
32        if (position_8002 < 1) {
33            System.err.print("Invalid position");
34        }
35        if (position_8002 == 1) {
36            NodeSLL_2511538002 new_node = new NodeSLL_2511538002 (value_8002);
37            new_node.next_8002 = head_8002;
38            return new_node;
39        }
40        else {
41            while (position_8002-- != 0) {
42                if (position_8002 == 0) {
43                    NodeSLL_2511538002 newNode = GetNode(value_8002);
44                    newNode.next_8002 = headNode_8002.next_8002;
45                    headNode_8002.next_8002 = newNode;
46                    break;
47                }
48                headNode_8002 = headNode_8002.next_8002;
49            }
50            if (position_8002 != 1) {
51                System.out.print("posisi di luar jangkauan");
52            }
53            return head_8002;
54        }
55    }
56    public static void printList(NodeSLL_2511538002 head_8002) {
57        NodeSLL_2511538002 curr = head_8002;
58        while (curr.next_8002 != null) {
59            System.out.print(curr.data_8002+"-->");
60            curr = curr.next_8002;
61        }
62        if (curr.next_8002 == null) {
63            System.out.print(curr.data_8002);
64        }
65        System.out.println();
66    }
67    public static void main(String[] args) {
68        // buat linked list 2->3->5->6
69        NodeSLL_2511538002 head_8002 = new NodeSLL_2511538002(2);
70        head_8002.next_8002 = new NodeSLL_2511538002(3);
71        head_8002.next_8002.next_8002 = new NodeSLL_2511538002(5);
72        head_8002.next_8002.next_8002.next_8002 = new NodeSLL_2511538002(6);
73        //catat list awal
74        System.out.print("Senarai berantai awal:");
75        printList(head_8002);
76        //tambahkan node baru di depan
77        System.out.print("tambah 1 simpul di depan: ");
78        int data_8002 = 1;
79        head_8002 = insertAtFront(head_8002, data_8002);
80        //catat update list
81        printList(head_8002);
82        //tambahkan node baru di belakang
83        System.out.print("tambah 1 simpul di belakang: ");
84        int data2 = 7;
85        head_8002 = insertAtEnd(head_8002, data2);
86        //catat update list
87        printList(head_8002);
88        System.out.print("tambah 1 simpul ke data 4: ");
89        int data3 = 4;
90        int pos=4;
91        head_8002 = insertPos(head_8002,pos,data3);
92        //catat update list
93        printList(head_8002);
94    }
95 }
```

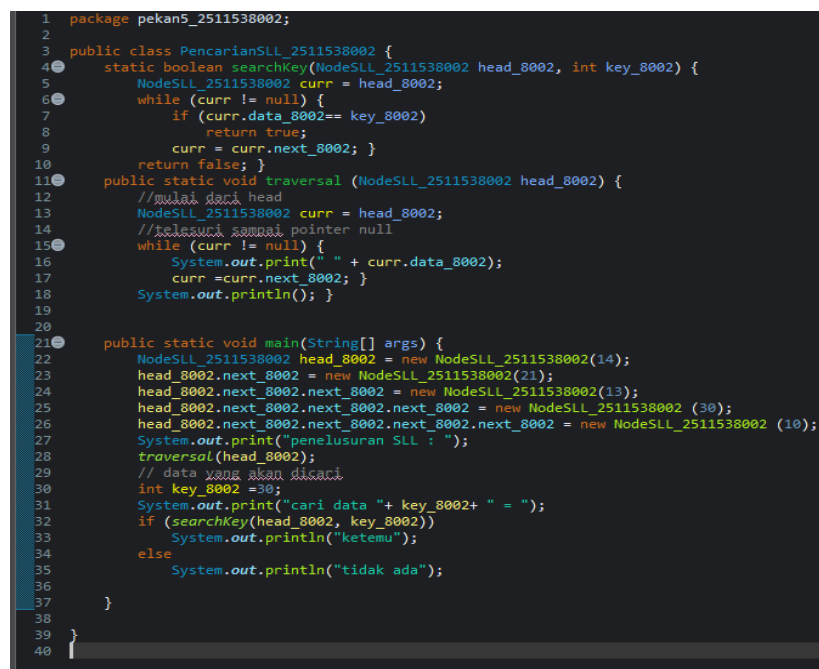
2. Jika semua yang Error diperbaiki lalu klik “Run” tuntuk melihat hasil dari syntax dari class TambahSLL_2511538002 :



```
<terminated> TambahSLL_2511538002 [Java Application]
Senarai berantai awal:2-->3-->5-->6
tambah 1 simpul di depan: 1-->2-->3-->5-->6
tambah 1 simpul di belakang: 1-->2-->3-->5-->6-->7
tambah 1 simpul ke data 4: 1-->2-->3-->5-->6-->7
```

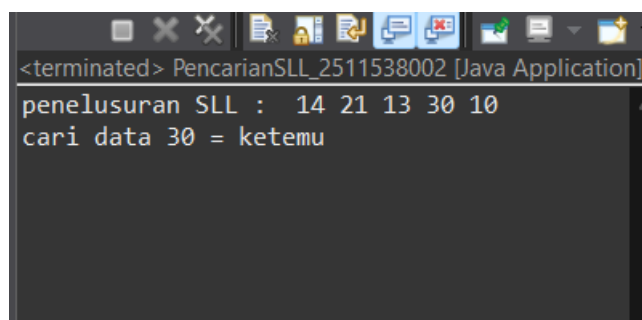
2.1.3 PencarianSLL_2511538002

1. Ulangi pembuatan Class sperti program sebelumnya dan beri nama Class tersebut “PencarianSLL_2511538002”. Lalu masukan Syntax seperti berikut:



```
1 package pekan5_2511538002;
2
3 public class PencarianSLL_2511538002 {
4     static boolean searchKey(NodeSLL_2511538002 head_8002, int key_8002) {
5         NodeSLL_2511538002 curr = head_8002;
6         while (curr != null) {
7             if (curr.data_8002== key_8002)
8                 return true;
9             curr = curr.next_8002; }
10        return false; }
11
12    public static void traversal (NodeSLL_2511538002 head_8002) {
13        //mulai dari head
14        NodeSLL_2511538002 curr = head_8002;
15        //telusuri sampai pointer null
16        while (curr != null) {
17            System.out.print(" " + curr.data_8002);
18            curr =curr.next_8002; }
19        System.out.println(); }
20
21    public static void main(String[] args) {
22        NodeSLL_2511538002 head_8002 = new NodeSLL_2511538002(14);
23        head_8002.next_8002 = new NodeSLL_2511538002(21);
24        head_8002.next_8002.next_8002 = new NodeSLL_2511538002(13);
25        head_8002.next_8002.next_8002.next_8002 = new NodeSLL_2511538002 (30);
26        head_8002.next_8002.next_8002.next_8002.next_8002 = new NodeSLL_2511538002 (10);
27        System.out.print("penelusuran SLL : ");
28        traversal(head_8002);
29        // data yang akan dicari
30        int key_8002 =30;
31        System.out.print("cari data "+ key_8002+ " = ");
32        if (searchKey(head_8002, key_8002))
33            System.out.println("ketemu");
34        else
35            System.out.println("tidak ada");
36    }
37
38 }
39
40 }
```

2. Kemudian klick “Run” untuk melhiat hasil dari Sytax “PencarianSLL_2511538002:



```
<terminated> PencarianSLL_2511538002 [Java Application]
penelusuran SLL : 14 21 13 30 10
cari data 30 = ketemu
```

2.1.4 HapusSLL_2511538002

1. Ulangi pembuatan Class seperti sebelumnya dan beri nama Class tersebut “HapusSLL_2511538002”. Lalu masukan Syntax seperti berikut.

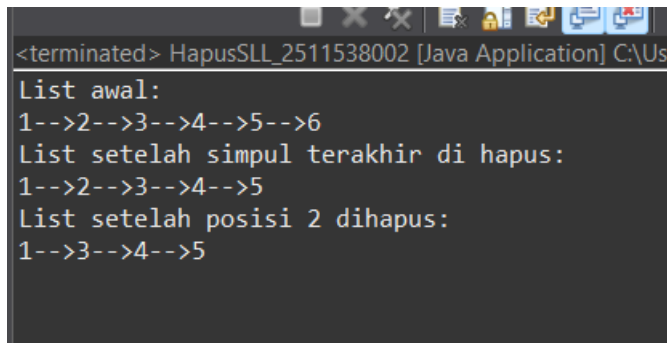
```

1 package pekan5_2511538002;
2
3 public class HapusSLL_2511538002 {
4     //fungsi untuk menghapus head
5     public static NodeSLL_2511538002 deleteHead (NodeSLL_2511538002 head_8002) {
6         // jika SLL kosong
7         if (head_8002 == null)
8             return null;
9         // pindahkan head ke node berikutnya
10        head_8002 = head_8002.next_8002;
11        //return head baru
12        return head_8002;
13    }
14
15    //fungsi menghapus node terakhir SLL
16    public static NodeSLL_2511538002 removeLastNode(NodeSLL_2511538002 head_8002) {
17        if (head_8002 == null) {
18            return null;
19        }
20        if (head_8002.next_8002 == null) {
21            return null;
22        }
23
24        NodeSLL_2511538002 secondLast = head_8002;
25        while (secondLast.next_8002.next_8002 != null) {
26            secondLast = secondLast.next_8002;
27        }
28
29        secondLast.next_8002 = null;
30        return head_8002;
31    }
32
33    // fungsi menghapus node di posisi tertentu
34    public static NodeSLL_2511538002 deleteNode (NodeSLL_2511538002 head_8002, int position_8002) {
35        NodeSLL_2511538002 temp = head_8002;
36        NodeSLL_2511538002 prev = head_8002;
37
38        if (temp == null)
39            return head_8002;
40
41        // kasus 1: head dihapus
42        if (position_8002 == 1) {
43            head_8002 = temp.next_8002;
44            return head_8002;
45        }
46
47        // kasus 2: hapus node tengah
48        for (int i = 1; temp != null && i < position_8002; i++) {
49            prev = temp;
50            temp = temp.next_8002;
51        }
52
53        if (temp != null) {
54            prev.next_8002 = temp.next_8002;
55        } else {
56            System.out.println("data tidak ada");
57        }
58
59        return head_8002;
60    }
61
62    // fungsi mencetak SLL
63    public static void printList(NodeSLL_2511538002 head_8002) {
64        NodeSLL_2511538002 curr = head_8002;
65        while (curr != null) {
66            System.out.print(curr.data_8002);
67            if (curr.next_8002 != null) {
68                System.out.print("-->");
69            }
70            curr = curr.next_8002;
71        }
72        System.out.println();
73    }
74
75    // kelas main
76    public static void main(String[] args) {
77        NodeSLL_2511538002 head_8002 = new NodeSLL_2511538002(1);
78        head_8002.next_8002 = new NodeSLL_2511538002(2);
79        head_8002.next_8002.next_8002 = new NodeSLL_2511538002(3);
80        head_8002.next_8002.next_8002.next_8002 = new NodeSLL_2511538002(4);
81        head_8002.next_8002.next_8002.next_8002.next_8002 = new NodeSLL_2511538002(5);
82        head_8002.next_8002.next_8002.next_8002.next_8002.next_8002 = new NodeSLL_2511538002(6);
83
84        System.out.println("List awal: ");
85        printList(head_8002);
86
87        head_8002 = removeLastNode(head_8002);
88        System.out.println("List setelah simpul terakhir di hapus: ");
89        printList(head_8002);
90
91        // deleting node at position 2
92        int position_8002 = 2;
93        head_8002 = deleteNode(head_8002, position_8002);
94
95        System.out.println("List setelah posisi 2 dihapus: ");
96        printList(head_8002);
97    }
98 }

```

2. dari Class ini sat saya pratikum ada beberapa yang Error, karena sala tulis, dan nama varaivelnyajuga sala, saya mengetik ulang lagi dari awal dan sekran

tidak, ada lagi Error. Kemudian kita klik “Run” untuk melihat hasil dari Syntax yang sudah dibuat:



```
<terminated> HapusSLL_2511538002 [Java Application] C:\Use
List awal:
1-->2-->3-->4-->5-->6
List setelah simpul terakhir di hapus:
1-->2-->3-->4-->5
List setelah posisi 2 dihapus:
1-->3-->4-->5
```

2.2 Tugas Pratikum pekan5_2511538002 .

2.2.1 Soal tugas pratikum

Pada tugas ini, mahasiswa diminta untuk mengimplementasikan konsep Single Linked List yang bekerja secara dinamis menggunakan pointer (referensi antar node). Tema tugas kali ini adalah Simulasi Antrian Rumah Sakit. Mahasiswa diwajibkan membuat program yang mensimulasikan sistem pendaftaran antrian pasien, di mana setiap pasien direpresentasikan sebagai sebuah node yang memiliki pointer ke pasien berikutnya. Pasien yang pertama kali mendaftar akan menjadi pasien pertama yang dipanggil (FIFO — First In, First Out).

2.2.2. Kelas RumahSakit_2511538002

1. ulangi pembuatan kelas seperti program sebelumnya dan beri nama Class tersebut “RumahSakit_2511538002” lalu masukan program seperti berikut:



```
package pekan5_2511538002;

public class Pasien_2511538002 {
    String Pasien_8002;
    String Penyakit_8002;
    int Antrian_8002;
    Pasien_2511538002 Next_8002;

    //konstruksi
    public Pasien_2511538002(String Pasien_8002, String Penyakit_8002, int Antrian_8002){
        this.Pasien_8002 = Pasien_8002;
        this.Penyakit_8002 = Penyakit_8002;
        this.Antrian_8002 = Antrian_8002;
        this.Next_8002 = null;
    }

    // getter
    public String getPasien_8002() {
        return Pasien_8002;
    }
    public String getPenyakit_8002() {
        return Penyakit_8002;
    }
    public int getAntrian_8002() {
        return Antrian_8002;
    }
    public Pasien_2511538002 getNext_8002() {
        return Next_8002;
    }

    // setter
    public void setNext_8002(Pasien_2511538002 Next_8002) {
        this.Next_8002 = Next_8002;
    }
}
```

2.2.3 Penjelasan kode program pada kelas RumahSakit_2511538002

1. package

```
package pekan5_2511538002;
```

Package digunakan untuk mengelompokkan file Java ke dalam satu folder atau proyek tertentu, sehingga program lebih rapi dan terorganisir. Pada program Pekan5_2511538002 adalah nama package tempat class disimpan:

2. Deklarasi Class

```
public class Pasien_2511538002 {
```

Class ini digunakan untuk mempresentasikan data pasien pada antrean rumah sakit.

3. Atribut

```
String Pasien_8002;  
String Penyakit_8002;  
int Antrian_8002;  
Pasien_2511538002  
Next_8002;
```

Bagian ini berisi atribut yang dimiliki setiap pasien :

- **Pasien** digunakan untuk menyimpan nama pasien.
- **Penyakit** digunakan untuk menyimpan informasi pasien.
- **Nomor antrian** digunakan untuk menyimpan nomor urut antrian pasien.
- **Next** digunakan untuk menunjukan node/pasien berikutnya dalam Single Linked List. Jika tidak ada pasien berikutnya nilainya null.

4. Konstruktor

```
Public Pasien_2511538002(String Pasien_8002,  
String Penyakit_8002,int Antrian_8002){  
this.Pasien_8002 = Pasien_8002;  
this.Penyakit_8002 = Penyakit_8002;  
this.Antrian_8002 = Antrian_8002;  
this.Next_8002 = null;
```

```
}
```

Konstruktor adalah metode khusus yang otomatis dipanggil saat objek dibuat. Konstruktor ini digunakan untuk mengisi data awal pasien, seperti

- mengisi atribut nama pasien.
- Mengisi atribut penyakit pasien.
- Mengisi nomor antrian pasien
- Mengatur pointer next menjadi kosong karena node baru belum terhubung ke node lain.

5. Getter

Getter digunakan untuk mengambil atau membaca data dari atribut

```
public String getPasien_8002() {  
    return Pasien_8002;  
}
```

Program ini digunakan untuk mengambil nama pasien.

```
public String getPenyakit_8002() {  
    return Penyakit_8002;  
}
```

Program ini digunakan untuk mengambil data penyakit pasien

```
public int getAntrian_8002() {  
    return Antrian_8002;  
}
```

Program ini digunakan untuk mengambil nomor antrian pasien.

```
public          Pasien_2511538002  
getNext_8002() {  
    return Next_8002;  
}
```

Program ini digunakan untuk mangambil alamat node berikutnya.

6. Setter

Setter digunakan untuk mengubah nilai atribut tertentu:

```
public void setNext_8002(Pasien_2511538002
Next_8002) {
    this.Next_8002 = Next_8002;
}
```

Fungsi dari program ini digunakan untuk menghubungkan node sekarang dengan node berikutnya artinya pasien 1 menunjukan ke pasien 2.

2.2.1. Kode Program RumahSakit_2511538002

```
1 package pekan5_2511538002;
2 import java.util.Scanner;
3
4 public class RumahSakit_2511538002 {
5     static Pasien_2511538002 head_8002 = null;
6     static int counter_8002 = 0;
7
8     // insert(daftar pasien)
9     public static void insertPasien_8002(String nama_8002, String penyakit_8002) {
10         counter_8002++;
11         Pasien_2511538002 baru = new Pasien_2511538002(nama_8002, penyakit_8002, counter_8002);
12
13         if (head_8002 == null) {
14             head_8002 = baru;
15         } else {
16             Pasien_2511538002 temp = head_8002;
17             while (temp.Next_8002 != null) {
18                 temp = temp.Next_8002;
19             }
20             temp.Next_8002 = baru;
21         }
22         System.out.println("pasien berhasil didaftarkan! nomor Antrian: " + counter_8002);
23     }
24
25     //delete head (hapusil pasien)
26     public static void panggilPasien_8002() {
27         if (head_8002 == null) {
28             System.out.println("Antrian kosong");
29             return;
30         }
31         System.out.println("memanggil pasien");
32         System.out.println("Nama: " + head_8002.Pasien_8002);
33         System.out.println("Keluhan: " + head_8002.Penyakit_8002);
34         System.err.println("No Antrian: " + head_8002.Antrian_8002);
35
36         head_8002 = head_8002.Next_8002;
37     }
38
39     //display
40     public static void tampilkan_8002() {
41         if (head_8002 == null) {
42             System.out.println("Antrian kosong!");
43             return;
44         }
45
46         Pasien_2511538002 temp = head_8002;
47         while (temp != null) {
48             System.out.println("No: " + temp.Antrian_8002 +
49                 "\n Nama: " + temp.Pasien_8002 +
50                 "\n Keluhan: " + temp.Penyakit_8002);
51             temp = temp.Next_8002;
52         }
53     }
54 }
```

```

//SEARCH
public static void cariPasien_8002(String Cari_8002) {
    Pasien_2511538002 temp = head_8002;
    boolean ditemukan = false;

    while (temp != null) {
        if (temp.Pasien_8002.equalsIgnoreCase(Cari_8002)) {
            System.out.println("Pasien ditemukan!");
            System.out.println("No: " + temp.Antrian_8002 +
                " | Nama: " + temp.Pasien_8002 +
                " | Keluhan: " + temp.Penyakit_8002);
            ditemukan = true;
            break;
        }
        temp = temp.Next_8002;
    }

    if (!ditemukan) {
        System.out.println("pasien tidak ditemukan!");
    }
}

//cek Status
public static void cekStatus_8002() {
    if (head_8002 == null) {
        System.out.println("Antrian Kosong!");
        return;
    }
    int jumlah = 0;
    Pasien_2511538002 temp = head_8002;

    while (temp != null) {
        jumlah++;
        temp = temp.Next_8002;
    }

    System.out.println("Jumlah pasien : " + jumlah);
    System.out.println("pasien terdepan: " + head_8002.Pasien_8002);
}

```

```

94
95 // main menu
96 public static void main(String[] args) {
97     Scanner input = new Scanner(System.in);
98     int pilihan;
99
100     do {
101         System.out.println("\n\n\n=== Antrian Rumah Sakit NIM: 2511538002 ===");
102         System.out.println("1. Daftarkan Pasien(Insert)");
103         System.out.println("2. Panggil pasien (Delete head)");
104         System.out.println("3. Tampilkan Antrian (Display)");
105         System.out.println("4. Cari Psien (Search)");
106         System.out.println("5. cek Status Antrian");
107         System.out.println("6. keluar");
108         System.out.print("Pilihan: ");
109         pilihan = input.nextInt();
110         input.nextLine();
111
112         switch (pilihan) {
113             case 1:
114                 System.out.print("Masukan Nama Pasien: ");
115                 String nama = input.nextLine();
116                 System.out.print("Keluhan: ");
117                 String keluhan = input.nextLine();
118                 insertPasien_8002(nama, keluhan);
119                 break;
120
121             case 2:
122                 panggilPasien_8002();
123                 break;
124
125             case 3:
126                 tampilkan_8002();
127                 break;
128
129             case 4:
130                 System.out.print("Masukan Nama: ");
131                 String cari = input.nextLine();
132                 cariPasien_8002(cari);
133                 break;
134
135             case 5:
136                 cekStatus_8002();
137                 break;
138
139             case 6:
140                 System.out.println("Terima Kasih!");
141                 break;
142
143             default:
144                 System.out.println("Pilihan Salah!");
145         }
146     } while (pilihan != 6);
147 }
148

```

2.2.2. Penjelasan kode kepada kelas program RumahSakit_2511538002

1 package dan Import

```
package pekan5_2511538002;  
import java.util.Scanner;
```

- **Package** digunakan untuk mengelompokkan file program dalam satu folder/proyek.
- **Scanner** digunakan sebagai alat untuk menerima input dari keyboard.

2 Deklarasi Class

```
public class RumahSakit_2511538002 {
```

Class ini merupakan class utama yang mengatur seluruh proses antrian rumah sakit.

3 Variabel Global

```
static Pasien_2511538002 head_8002 = null;  
static int counter_8002 = 0;
```

- **Head_8002** merupakan pointer awal pada Single Linked List. Node pertama pada antrean akan disimpan pada variabel ini. nilai awal null berarti antrian masih kosong.
- **Counter** variabel untuk menyimpan jumlah pasien sekaligus nomor antrean otomatis.

4 Method Insert Pasien

```
public static void insertPasien_8002(String  
nama_8002, String penyakit_8002) {  
    counter_8002++;  
    Pasien_2511538002 baru = new  
    Pasien_2511538002(nama_8002,  
    penyakit_8002, counter_8002);  
    if (head_8002 == null) {  
        head_8002 = baru;  
    } else {
```

```

        Pasien_2511538002 temp = head_8002;
        while (temp.Next_8002 !=null) {
            temp = temp.Next_8002;
        }
        temp.Next_8002 = baru;
    }

    System.out.println("pasien berhasil
    didaftarkan! nomor Antrian: " +
    counter_8002);
}

```

Method InsertPasien_8002() digunakan untuk membuat node pasien baru, menabahkan nomor antrian secara otomatis, lalu menghubungkan node tersebut ke bagian akhir Linked list menggunakan proses traversal.

5 Method Delete Head

```

public static void panggilPasien_8002() {
    if (head_8002 == null) {
        System.out.println("Antrian kosong");
        return;
    }
    System.out.println("memanggil pasient");
    System.out.println("Nama:          "          +
    head_8002.Pasien_8002);
    System.out.println("Keluhan:          "+"
    head_8002.Penyakit_8002);
    System.err.println("No      Antrian:      "      +
    head_8002.Antrian_8002);
    head_8002 = head_8002.Next_8002;
}

```

Method PanggilPasien_8002() digunakan untuk memproses pasien paling depan pada antrain dengan megakses node pertama (head) lalu memindahkan pointer head ke node berikutnya

6 Method Display

```
public static void tampilkan_8002() {
    if (head_8002 == null) {
        System.out.println("Antrian kosong!");
        return;
    }
    Pasien_2511538002 temp = head_8002;
    while (temp != null) {
        System.out.println("No: " + temp.Antrian_8002 +
                           "| Nama: " + temp.Pasien_8002 +
                           "| Keluhan: " +
                           temp.Penyakit_8002);
        temp = temp.Next_8002;
    }
}
```

Method tampilkan_8002() digunakan untuk menampilkan seluruh data pasien yang terdapat di dalam antrian. Proses dilakukan dengan melakukan traversal atau penelusuran dari node pertama hingga node terakhir menggunakan variabel sementara bernama temp. Selama node masih ada (temp != null), data pasien seperti nomor antrean, nama seluruh isi linked list secara berurutan.

7 Method Search

```
public static void cariPasien_8002(String
Cari_8002) {
    Pasien_2511538002 temp = head_8002;
    boolean ditemukan = false;
    while (temp != null) {
```



```

    if
    (temp.Pasien_8002.equalsIgnoreCase(Cari_8002
    )) {
    System.out.println("Pasien ditemukan!");
    System.out.println("No: " +
    temp.Antrian_8002 +
        "Nama: " + temp.Pasien_8002 +
        " | Keluhan: "
        temp.Penyakit_8002);
    ditemukan = true;
    break;
    }
    temp = temp.Next_8002;
    }
    if (!ditemukan) {
    System.out.println("pasien tidak
    ditemukan!");
    }
    }

```

Method Cari pasien_8002() digunakan untuk melakukan pencarian nama pasien pada Linked list dengan cara menelusuri node satu per satu dan membandingkan nama pasien menggunakan equalsIgnoreCase().

8 Method cek Status

```

public static void cekStatus_8002() {
    if (head_8002 == null) {
    System.out.println("Antrian Kosong!");
    return;
    }
    int jumlah = 0;
    Pasien_2511538002 temp = head_8002;
    while (temp != null) {
    jumlah++;
    }
}

```

```

temp = temp.Next_8002;
}
System.out.println("Jumlah pasien : " +
jumlah);
System.out.println("pasien terdepan: " +
head_8002.Pasien_8002);
}

```

Method cekStatus_8002() digunakan untuk membaca kondisi antrian saat ini dengan menghitung jumlah node pada linked List dan menampilkan pasien yang berada di posisi terdepan

9 Method Main

```

public static void main(String[] args) {
Scanner input = new Scanner(System.in);
int pilihan;
do {
System.out.println("\n\n\n=== Antrian Rumah
Sakit NIM: 2511538002 ===");
System.out.println("1.                Daftarkan
Pasien(Insert)");
System.out.println("2.        Panggil        pasien
(Delete head)");
System.out.println("3.    Tampilkan    Antrian
(Display)");
System.out.println("4.            Cari            Psien
(Search)");
System.out.println("5.            cek            Status
Antrian");
System.out.println("6. keluar");
System.out.print("Pilihan: ");
pilihan = input.nextInt();
input.nextLine();
switch (pilihan) {

```

```
case 1:
System.out.print("Masukan Nama Pasien: ");
String nama = input.nextLine();
System.out.print("Keluhan: ");
String keluhan = input.nextLine();
insertPasien_8002(nama, keluhan);
break;
```

Method Main() merupakan bagian utama program yang pertama kali dijalankan dan berisi pengaturan menu, input pengguna, perulangan program, serta pemanggilan method sesuai pilihan menu.

10 Case 1

```
case 1:
    System.out.print("Masukan Nama Pasien: ");
    String nama = input.nextLine();
    System.out.print("Keluhan: ");
    String keluhan = input.nextLine();
    insertPasien_8002(nama, keluhan);
    break;
```

Pada bagian ini digunakan untuk proses pendaftaran pasien.

11 Case 2

```
case 2:
panggilPasien_8002();
break;
```

Case 2 digunakan untuk proses pemanggilan pasien.

12 case 3

```
case 3:
tampilkan_8002();
break;
```

Case 3 bagian ini digunakan untuk menampilkan seluruh antrian

13 case 4

```
case 4:
    System.out.print("Masukan Nama: ");
    String cari = input.nextLine();
    cariPasien_8002(cari);
    break;
```

Case 4 digunakan untuk pencarian pasien.

14 case 5

```
case 5:
    cekStatus_8002();
    break;
```

Case 5 ini digunakan untuk pengecekan status antrian.

15 case 6

```
case 6:
    System.out.println("Terima Kasih!");
    break;
```

Case 6 digunakan untuk mengakhiri program

16 default

```
default:
    System.out.println("Pilihan Salah!");
}
```

Bagian default digunakan untuk menangani kondisi ketika pengguna memasukkan pilihan menu yang tidak tersedia.

17 Penutu perulangan

```
while (pilihan != 6);
{
}
}
```

Bagian While (pilihan != 6) digunakan sebagai syarat agar program terus berjalan selama pengguna belum memilih menu keluar.

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan hasil praktikum dan pembuatan program simulasi antrean rumah sakit menggunakan struktur data single-linked list, dapat disimpulkan bahwa struktur data linked list mampu digunakan untuk menyimpan dan mengelola data pasien secara dinamis. Setiap data pasien dipresentasikan sebagai node yang saling terhubung menggunakan pointer menuju node berikutnya.

Pada program ini telah diterapkan beberapa operasi dasar Single Linked List, yaitu insert untuk menambahkan pasien ke dalam antrean, delete head untuk memanggil pasien terdepan, display untuk menampilkan seluruh data pasien, search untuk mencari data pasien berdasarkan nama, serta pengecekan status antrean. Program juga menggunakan konsep traversal untuk menelusuri seluruh node pada linked list.

DAFTAR PUSTAKA

Struktur Data Menggunakan Java. *Struktur Data Menggunakan Java*. Andi Publisher, 2019.

Data Structures and Algorithms in Java. *Data Structures and Algorithms in Java*. Wiley, 2014.